The ESP32 Classic Field Guide

Nov 13, 2026 · @Chuck Phillips

Introduction: meet your ESP32

Your NodeMCU-style board carries the original ESP32 (the "classic" ESP32-D0WD chip inside an ESP32-WROOM-32 module): two 240 MHz cores, Wi-Fi, Bluetooth Classic and BLE, and about 34 usable-or-not GPIO pads, for a few dollars. Espressif launched it in 2016, and newer chips (S2, S3, C3, C6, H2, P4) have since branched off, but the classic is still the only one with Bluetooth Classic (the kind used for A2DP audio and older gamepads), and the only Wi-Fi ESP32 with a built-in Ethernet MAC.

This book walks from the silicon outward: the chip and its CPU, the radios, the board you hold, every pin and what it can do, the hardware blocks behind the pins, power and sleep, security, how to program it, and a pile of project ideas.

Which board is mine? Two NodeMCU-style layouts are common:

Board	Pins	USB chip	Onboard LED	How to tell
ESP32 DevKit V1 (DOIT)	30	CP2102 or CH340	Blue LED on GPIO 2	15 pins per side, no GPIO 6–11 broken out
NodeMCU-32S / DevKitC	38	CP2102 or CH340	Varies (often none or GPIO 2)	19 pins per side, includes CMD/CLK/SD0–SD3 (flash pins)

Everything in this book applies to both; the 38-pin board simply breaks out the six flash pins and a couple of extra GPIOs (0 and 2 are on both).

Chapter 1: The chip at a glance

The classic ESP32 is a dual-core 32-bit microcontroller with 520 KB of RAM, 2.4 GHz Wi-Fi and Bluetooth 4.2 on one 5×5 or 6×6 mm chip, plus a big set of hardware peripherals that can be routed to almost any pin (ESP32 Series Datasheet).

Feature	ESP32 classic (D0WD-V3 in WROOM-32E)
CPU	2× Xtensa LX6, 32-bit, up to 240 MHz (about 600 DMIPS total)
Low-power co-CPU	ULP coprocessor (FSM), runs while main CPUs sleep
ROM	448 KB (boot loader, core libraries)
SRAM	520 KB on chip
RTC memory	16 KB (8 KB fast + 8 KB slow), survives deep sleep
Flash	External SPI flash, 4 MB typical (4/8/16 MB on WROOM-32E)
PSRAM	None on WROOM; up to 4 MB usable on WROVER modules
Wi-Fi	802.11 b/g/n, 2.4 GHz, up to 150 Mbps, STA / AP / both
Bluetooth	v4.2 BR/EDR (Classic) + Bluetooth LE
GPIO	34 physical GPIOs (about 25 safe to use on a dev board)
ADC	2× 12-bit SAR ADCs, 18 channels
DAC	2× 8-bit (GPIO 25, 26)
Touch	10 capacitive touch inputs
UART	3
SPI	4 (2 used for flash, HSPI and VSPI free)
I²C	2
I²S	2 (audio in/out, also drives parallel LCDs and cameras)
PWM	LED PWM 16 channels + 2 motor PWM units
Other	RMT (IR/LED timing), pulse counter, SD/SDIO host, Ethernet MAC, CAN (TWAI), hall sensor

Feature	ESP32 classic (D0WD-V3 in WROOM-32E)
Security	Secure boot, flash encryption, 1024-bit eFuse, AES/SHA/RSA/RNG accelerators
Supply	3.0–3.6 V (3.3 V nominal); pins are 3.3 V only, not 5 V tolerant
Temperature	−40 to +85 °C (WROOM-32E, 105 °C versions exist)

The one number to remember: **3.3 V logic**. Never put 5 V on a GPIO; use a level shifter or a resistor divider.

Chapter 2: The CPU, memory and clocks

The ESP32 is really three processors: two full Xtensa LX6 cores for your code and the radios, and a tiny ULP coprocessor that keeps watch while the big cores sleep.

The two main cores

- **PRO_CPU (core 0)** and **APP_CPU (core 1)**. Both are identical 32-bit Xtensa LX6 cores running at 80, 160 or 240 MHz.
- Each core has a single-precision **floating-point unit**, so `float` math is fast; `double` is done in software and is several times slower.
- Xtensa uses **windowed registers**: 64 physical registers, 16 visible at a time, which makes function calls cheap.
- Code runs under **FreeRTOS** (dual-core SMP). In the Arduino core, the Wi-Fi and Bluetooth stacks live mostly on core 0 and your `loop()` runs on core 1, so heavy work in `loop()` doesn't starve the radio.
- You can pin your own tasks to a core with `xTaskCreatePinnedToCore()`: a classic trick is VGA or audio generation on one core and logic on the other (the same split your TidyOS VGA build benefits from).

The ULP coprocessor

The Ultra-Low-Power coprocessor is a simple state-machine CPU that lives in the RTC domain. It runs small programs from **8 KB of RTC slow memory** while the main cores are in deep sleep. It can read the ADC, touch pads, RTC GPIOs and an RTC I²C bus, then wake

the main CPU only when something interesting happens (for example, "wake me when the temperature passes 30 °C").

Memory map

Region	Size	What it's for
Internal ROM	448 KB	Boot loader, low-level libraries, can't be changed
SRAM0	192 KB	Instruction RAM (IRAM) for fast code, plus flash cache
SRAM1	128 KB	Data or instruction RAM
SRAM2	200 KB	Data RAM (DRAM): heap, stacks, buffers
RTC fast	8 KB	Code that runs right after wake-up
RTC slow	8 KB	ULP program and data that survives deep sleep (`RTC_DATA_ATTR`)
External flash	4 MB typical	Your program (run in place through the cache), files (SPIFFS/LittleFS), OTA slots

In practice, an Arduino sketch with Wi-Fi running has roughly 150–250 KB of heap free. Big frame buffers (like a 640×480 VGA screen) need clever tricks such as 3-bit color or line-by-line drawing, because the whole SRAM is only 520 KB.

Clocks and timers

- **40 MHz crystal** feeds a PLL that makes the 240 MHz CPU clock and the 80 MHz APB bus clock for the peripherals.
- **Audio PLL (APLL)** gives exact audio sample rates for I²S.
- **RTC clocks:** an internal ~150 kHz RC oscillator (or an optional 32.768 kHz crystal on GPIO 32/33) keeps time during sleep.
- **Timers:** four 64-bit general-purpose timers (two groups of two), plus three watchdogs that reset the chip if your code locks up.

Changing CPU speed is one line in Arduino: `setCpuFrequencyMhz(80);` cuts power use noticeably (Wi-Fi needs at least 80 MHz).

Chapter 3: The radios

One 2.4 GHz radio and one antenna are shared by Wi-Fi and Bluetooth; the chip time-slices between them, so both work at once but each gets a little slower.

Wi-Fi (802.11 b/g/n)

- **Modes:** station (joins your router), soft access point (makes its own network, up to about 10 clients), or both at once (AP+STA, great for setup portals and repeaters).
- **Speed:** up to 150 Mbps link rate on paper; real TCP throughput is typically 10–20 Mbit/s.
- **Range:** the WROOM PCB antenna is good for a house; the WROOM-32UE / IPEX versions take an external antenna for much more.
- **Extras:** promiscuous (sniffer) mode, raw 802.11 frame injection, Wi-Fi Protected Setup, WPA3, and **ESP-NOW**, Espressif's router-free peer-to-peer protocol (250-byte packets, low latency, hundreds of meters line-of-sight).
- **Fine Timing / FTM** is not on the classic chip; that arrived with later chips.

Bluetooth Classic (BR/EDR)

The classic ESP32 is the only current ESP32 that speaks Bluetooth Classic. That unlocks:

- **A2DP** audio: be a Bluetooth speaker (sink) or send music to headphones (source).
- **SPP** serial: a wireless UART to a phone or PC.
- **HID host:** read older Bluetooth keyboards, mice and game controllers (PS3/PS4/Wii controllers via libraries like Bluepad32).
- **HFP/AVRCP** for hands-free calling and track controls.

Bluetooth Low Energy (4.2)

- GATT server and client, advertising and scanning, iBeacon/Eddystone beacons.
- **BLE HID device:** the ESP32 can pretend to be a keyboard, mouse or gamepad.
- **BLE Mesh** is supported in ESP-IDF.

Tip: Wi-Fi + Bluetooth Classic + your app together can be tight on RAM; if you only need BLE, the NimBLE stack saves around 50–100 KB.

Chapter 4: Board tour and pinout map

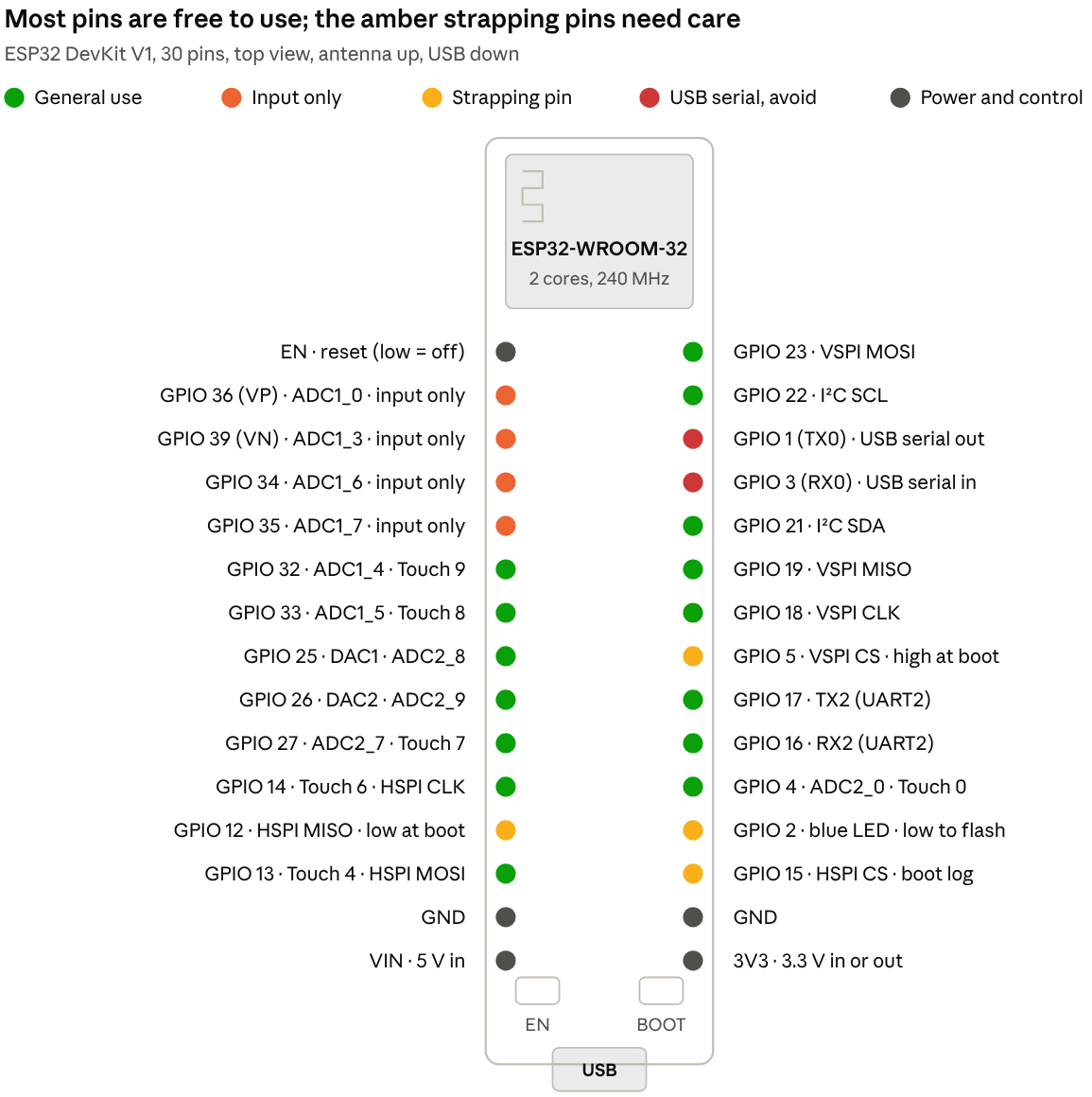
The dev board is just the ESP32 module plus the parts needed to power it and program it over USB.

Part on the board	What it does
ESP32-WROOM-32 module (metal can)	The chip, 4 MB flash, 40 MHz crystal and the zig-zag PCB antenna
Micro-USB or USB-C jack	5 V power and serial programming
CP2102 or CH340 chip	USB-to-serial bridge (Windows may need its driver)
AMS1117-3.3 regulator	Turns 5 V into 3.3 V, about 600–800 mA usable
EN button	Reset (pulls the chip-enable pin low)
BOOT button	Holds GPIO 0 low to enter download mode
Red LED	Power on
Blue LED	GPIO 2 (DevKit V1); your "hello world" blink
Two transistors	Auto-reset: let the PC flash the board without pressing buttons

Power pins: feed 5 V into **VIN** (or USB), or a clean regulated 3.3 V into **3V3**. Never feed both 3V3 and USB at once from different supplies. The 3V3 pin can power small sensors and displays; budget about 300 mA for them while Wi-Fi is running.

If uploads fail with "Failed to connect": hold BOOT, tap EN, release BOOT, then upload. A 10 µF capacitor between EN and GND makes this automatic on boards with flaky auto-reset.

The map below is the common 30-pin DevKit V1, seen from the top with the antenna up and USB down. Silkscreen labels like D23 just mean GPIO 23; VP and VN are GPIO 36 and 39.



ESP32 DevKit V1 pinout · 30 pins, colour-coded by how safe they are to use

Green pins are the ones to reach for first. Orange pins can only read, amber (yellow) pins change how the chip boots, and red pins are busy talking to your PC over USB.

Chapter 5: Pin-by-pin guide

Thanks to the **GPIO matrix**, almost any digital function (UART, SPI, I²C, PWM, I²S, RMT, pulse counter) can be routed to almost any output-capable pin. What's fixed to specific pins are the analog features (ADC, DAC, touch), deep-sleep wake-up (RTC GPIOs) and the boot "strapping" behavior. This table tells you exactly that for every pin.

Rules that apply to every pin: 3.3 V logic only; about 20 mA per pin is a comfortable limit (40 mA absolute max); internal pull-up and pull-down resistors (~45 kΩ) exist on all pins except 34–39.

GPIO	Out?	ADC	Touch	Deep-sleep wake (RTC)	Watch out for	Good for
0	Yes	ADC2_1	T1	Yes	Strapping: LOW at boot = flash mode (BOOT button). Not on 30-pin board	Button input (it already is one)
1 (TX0)	Yes	—	—	No	USB serial out; boot messages appear here	Leave alone
2	Yes	ADC2_2	T2	Yes	Strapping: must be LOW/floating to flash. Drives blue LED	Status LED, light outputs
3 (RX0)	Input*	—	—	No	USB serial in; goes HIGH at boot	Leave alone
4	Yes	ADC2_0	T0	Yes	None	Touch button, any I/O, DHT sensor
5	Yes	—	—	No	Strapping (SDIO timing); outputs PWM briefly at boot	VSPI CS for SD cards and displays
6–11	—	—	—	No	Wired to the SPI flash. Using them crashes the chip	Never use
12	Yes	ADC2_5	T5	Yes	Strapping: HIGH at boot selects 1.8 V flash and boot fails	Outputs that are off at boot, HSPI MISO
13	Yes	ADC2_4	T4	Yes	None	HSPI MOSI, touch, any I/O
14	Yes	ADC2_6	T6	Yes	Outputs PWM briefly at boot	HSPI CLK, touch
15	Yes	ADC2_3	T3	Yes	Strapping: LOW at boot silences boot log; PWM at boot	HSPI CS, touch
16	Yes	—	—	No	Used for PSRAM on WROVER modules	UART2 RX, LED strips, any I/O
17	Yes	—	—	No	Used for PSRAM on WROVER modules	UART2 TX, any I/O
18	Yes	—	—	No	None	VSPI CLK (SD card, TFT)
19	Yes	—	—	No	None	VSPI MISO

GPIO	Out?	ADC	Touch	Deep-sleep wake (RTC)	Watch out for	Good for
21	Yes	—	—	No	None	I²C SDA (default)
22	Yes	—	—	No	None	I²C SCL (default)
23	Yes	—	—	No	None	VSPI MOSI
25	Yes	ADC2_8	—	Yes	None	**DAC1**: audio out, analog voltage
26	Yes	ADC2_9	—	Yes	None	**DAC2**: audio out, analog voltage
27	Yes	ADC2_7	T7	Yes	None	Touch, any I/O
32	Yes	ADC1_4	T9	Yes	Optional 32 kHz crystal	Analog sensors (works with Wi-Fi on)
33	Yes	ADC1_5	T8	Yes	Optional 32 kHz crystal	Analog sensors (works with Wi-Fi on)
34	No	ADC1_6	—	Yes	Input only, no pull resistors	Potentiometers, battery voltage
35	No	ADC1_7	—	Yes	Input only, no pull resistors	Analog sensors
36 (VP)	No	ADC1_0	—	Yes	Input only, no pull resistors	Analog sensors, microphone
37, 38	—	—	—	—	Not brought out on WROOM modules	—
39 (VN)	No	ADC1_3	—	Yes	Input only, no pull resistors	Analog sensors

GPIO 20, 24 and 28–31 don't exist on this chip.

The three rules that save the most headaches

1. **Use ADC1 (GPIO 32–39) for analog when Wi-Fi is on.** ADC2 is shared with the Wi-Fi driver, and `analogRead()` on ADC2 pins fails while Wi-Fi runs.
2. **Keep strapping pins (0, 2, 5, 12, 15) in their default state at power-up.** A pull-up on GPIO 12 or a sensor holding GPIO 2 high is the classic "my board won't boot / won't flash" bug.
3. **Input-only pins (34–39) need external pull-up resistors** for buttons, because they have none built in.

Best "first choice" pins

For plain digital in/out with no surprises: **4, 13, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33**. That's 14 worry-free pins, more than most projects need.

Chapter 6: The hardware blocks behind the pins

The pins are just doors; the real power is the dozen hardware blocks behind them, which do timing-critical work by themselves so your code doesn't have to.

Block	What it does	Which pins	Things you can build
GPIO	Digital in/out, interrupts on any edge	Any (34–39 input only)	Buttons, relays, reed switches, door sensors
ADC (2× 12-bit, 18 ch)	Reads 0–~3.1 V as 0–4095; set attenuation per pin	32–39 (ADC1), 0/2/4/12–15/25–27 (ADC2)	Battery monitor, light/soil/gas sensors, joysticks, potentiometers
DAC (2× 8-bit)	Outputs a real analog voltage, 0–3.3 V in 256 steps	25, 26 only	8-bit audio, sine/tone generator, composite video (yes, really)
Touch (10 pads)	Senses a finger through a wire, foil or PCB pad	0, 2, 4, 12–15, 27, 32, 33	Buttonless keypads, touch lamp, hidden switches, wake on touch
LED PWM (LEDC, 16 ch)	Hardware PWM up to 40 MHz, up to 20-bit resolution	Any output pin	LED dimming, servos, fan speed, buzzers
Motor PWM (MCPWM, 2 units)	Paired PWM with dead time, fault input, capture	Any output pin	Brushless/brushed motor drivers, H-bridges, robots
RMT (8 ch)	Sends and reads precisely timed pulse trains	Any output pin	IR remote send/receive, WS2812/NeoPixel LED strips, 433 MHz RF codes

Block	What it does	Which pins	Things you can build
Pulse counter (PCNT, 8 units)	Counts edges in hardware, even fast ones	Any input pin	Rotary encoders, flow meters, anemometers, frequency counter
UART (3)	Serial ports, up to 5 Mbaud	Any (UART0 = 1/3 for USB)	GPS modules, Meshtastic/LoRa boards, RS-485 with a transceiver
I²C (2)	Two-wire bus, 100 kHz–1 MHz, many devices	Any (default 21 SDA, 22 SCL)	OLEDs, BME280, IMUs, RTCs, port expanders
SPI (HSPI + VSPI)	Fast 4-wire bus, up to 80 MHz	Any; native pins 12–15 and 5/18/19/23 are fastest	SD cards, TFT displays, LoRa radios, W5500 Ethernet
I²S (2)	Digital audio in/out; also parallel/camera mode	Any	Internet radio with a MAX98357 amp, INMP441 mic, DMA-driven VGA
SD/SDIO host	1-bit or 4-bit SD card at high speed	Fixed: 2, 4, 12, 13, 14, 15	Fast data logging, media player, file server
Ethernet MAC (RMII)	Wired 10/100 Ethernet with an external PHY	Fixed RMII pins (0, 19, 21, 22, 25, 26, 27, plus MDC/MDIO)	PoE-powered sensors (Olimex/WT32-ETH01-style)
TWAI (CAN 2.0)	Car and industrial CAN bus, with a transceiver chip	Any	OBD-II car data reader, CAN sniffer
Hall sensor	Detects magnetic fields inside the chip	Internal (uses 36/39)	Magnet-triggered switch (dropped in newer ESP-IDF 5 releases)
Timers and watchdogs	4× 64-bit timers, 3 watchdogs	Internal	Precise scheduling, auto-recovery from lockups

Some favorite tricks

- **VGA output with resistors:** I²S or a fast GPIO loop plus a few resistors make 640×480 VGA in a few colors (the FabGL and Bitluni libraries).
- **Composite TV video** straight from the DAC on GPIO 25, no extra parts.
- **Software-defined Bluetooth speaker:** A2DP sink + I²S amp + speaker in about 20 lines of code.
- **NeoPixels by the thousand:** RMT or I²S DMA drive long LED strips with almost no CPU use (WLED runs on exactly this chip).
- **Wireless sniffer:** Wi-Fi promiscuous mode + BLE scanning for packet counting and surveys.
- **Retro emulators:** NES, Game Boy, ZX Spectrum and C64 emulators exist for the classic ESP32.

Chapter 7: Power and sleep

The chip can go from about 240 mA while transmitting Wi-Fi down to about 10 µA in deep sleep, a 24,000× range, which is what makes battery projects possible.

Mode	What's running	Chip current (approx.)
Wi-Fi transmitting	Everything, radio sending	~240 mA peaks
Wi-Fi receiving / active	CPUs + radio listening	~95–100 mA
Modem sleep	CPUs on, radio off between beacons	~20–68 mA (depends on MHz)
Light sleep	CPUs paused, RAM kept, quick wake	~0.8 mA
Deep sleep + ULP	Only RTC domain and ULP running	~150 µA
Deep sleep	RTC timer + RTC memory	~10 µA
Hibernation	RTC timer only	~5 µA

Source: ESP32 Series Datasheet, power consumption section.

The dev-board catch: these are chip numbers. Your NodeMCU board's AMS1117 regulator, USB chip and red power LED draw several milliamps on their own, so deep sleep on the dev board is closer to 5–15 mA. For a real battery build, use a bare module or a low-power board (FireBeetle, TinyPICO style) with an LDO like the HT7333.

Ways to wake up

- **Timer:** "sleep 10 minutes, then measure and send."
- **ext0:** one RTC GPIO going high or low (a button or PIR sensor).
- **ext1:** several RTC GPIOs at once (any high, or all low).
- **Touch pad:** touch a wire to wake.
- **ULP coprocessor:** wake when a sensor reading crosses a threshold.
- **Light sleep only:** any GPIO, or UART activity.

Deep sleep restarts your program from `setup()`. Keep counters or state across sleeps with the `RTC_DATA_ATTR` keyword, which stores a variable in the 8 KB of RTC slow memory.

Powering it

- **USB or VIN:** 5 V (about 4.5–12 V is tolerated by most AMS1117 boards, but 5–7 V keeps the regulator cool).
- **3V3 pin:** a clean 3.0–3.6 V supply, bypassing the regulator. A single LiFePO4 cell (3.2 V) works directly.
- **Wi-Fi brownouts:** the 240 mA transmit bursts can reset a weakly powered board. Add a 100–470 µF capacitor across 3V3 and GND and use a good USB cable.

Chapter 8: Security features

The ESP32 can lock down its firmware so it can't be read or replaced, but most of these switches are one-way eFuses, so experiment on a spare board.

Feature	What it does	Reversible?
Hardware crypto	AES, SHA-1/SHA-2, RSA and big-number math accelerators speed up HTTPS and TLS	Always on

Feature	What it does	Reversible?
True random number generator	Real random numbers from radio noise (`esp_random()`) when Wi-Fi or BT is on	Always on
eFuse (1024 bits)	One-time-programmable bits: MAC address, keys, chip config	No, burned forever
Flash encryption	Encrypts everything in flash with a key hidden in eFuse, so a copied flash chip is useless	Development mode can be turned off a few times; release mode is permanent
Secure boot	Only boots firmware signed with your key	No
JTAG disable	Turns off the debug port	No

Every ESP32 also has a unique factory **MAC address** burned into eFuse, handy as a device ID (`ESP.getEfuseMac()`).

For hobby projects you'll rarely need more than HTTPS (which the crypto blocks make fast) and keeping Wi-Fi passwords out of your GitHub repos. If you ever sell a gadget, flash encryption plus secure boot keeps people from cloning your firmware.

Chapter 9: Ways to program it

The classic ESP32 has the widest software support of any ESP chip, from one-click firmwares to bare-metal C.

Option	Language	Best for
Arduino IDE (esp32 core)	C++	Quick projects, huge library collection
PlatformIO (in VS Code)	C++	Bigger Arduino or ESP-IDF projects, versioned libraries
ESP-IDF	C / C++	Espressif's official SDK: full control of FreeRTOS, both cores, every peripheral
MicroPython	Python	Interactive tinkering at a prompt, fast experiments

Option	Language	Best for
ESPHome	YAML	Home Assistant sensors and switches with no code
Tasmota	Web config	Smart plugs, relays, sensors with MQTT
WLED	Web config	Addressable LED strips
Rust (esp-rs)	Rust	Memory-safe firmware
Espruino / Lua / Toit	JavaScript / Lua / Toit	Scripting alternatives

Flashing tools

- **esptool** (Python command line): the tool everything else uses underneath.
- **ESP Web Tools / browser flashers:** flash from Chrome or Edge over Web Serial, no install. This is how your GitHub Pages web flashers work.
- **OTA updates:** once running, the board can update itself over Wi-Fi (needs a partition scheme with two app slots).

Your first sketch

```
// Blink the blue LED and say hello
void setup() {
  Serial.begin(115200);
  pinMode(2, OUTPUT);
}

void loop() {
  digitalWrite(2, HIGH);
  delay(500);
  digitalWrite(2, LOW);
  delay(500);
  Serial.printf("Running on core %d at %d MHz\n", xPortGetCoreID(),
getCpuFrequencyMhz());
}
```

In Arduino IDE, pick the board **"ESP32 Dev Module"** (or "DOIT ESP32 DEVKIT V1"), the right COM port, and upload. Open Serial Monitor at 115200 baud.

Partition schemes

Your 4 MB of flash is split into app slots, a file system and settings storage (NVS). Under Tools → Partition Scheme you can trade OTA for space: **Default** (2 × 1.2 MB apps + 1.5 MB files), **Huge APP** (3 MB app, no OTA, for big Bluetooth + Wi-Fi sketches) or **Minimal SPIFFS** (2 × 1.9 MB apps).

Chapter 10: Project ideas and a gotchas cheat sheet

Here are projects that use the classic ESP32's strengths, with a suggested pin plan for each so they won't fight the strapping pins.

Project	What it uses	Suggested pins	Level
Bluetooth speaker	BT Classic A2DP + I²S amp (MAX98357)	BCLK 26, LRC 25, DIN 22	Easy
Internet radio	Wi-Fi + I²S DAC (PCM5102) + OLED	I²S 26/25/22, OLED on I²C SDA 21, SCL 33	Medium
Touch-controlled lamp	Touch pads + LED PWM + MOSFET	Touch 4, 27; PWM 23	Easy
Weather station on battery	Deep sleep + BME280 + ADC battery check	I²C 21/22, battery on 34 via divider	Medium
NeoPixel light show (WLED)	RMT/I²S driving WS2812 strips	Data 16	Easy
IR universal remote	RMT send and receive	IR LED 4, receiver 35	Medium
ESP-NOW walkie-talkie network	ESP-NOW + I²S mic (INMP441) + I²S amp	Mic 32/33/27, amp 26/25/22	Hard
Retro game console	VGA via resistors or composite via DAC + gamepad over BT	VGA 18/19/21/22/23, or composite 25	Hard
Car OBD-II reader	TWAI (CAN) + SN65HVD230 transceiver	TX 17, RX 16	Medium
Wi-Fi / BLE survey logger	Promiscuous mode + BLE scan + SD card	SD on VSPI 18/19/23/5	Medium

Project	What it uses	Suggested pins	Level
Rotary-encoder control knob	Pulse counter + BLE HID	Encoder 32/33, button 27	Easy
Data logger with web dashboard	Async web server + LittleFS + any sensors	I²C 21/22, analog 36/39	Medium

Gotchas cheat sheet

- [ ] Never use GPIO 6–11 (flash).
- [ ] Nothing pulling GPIO 12 high at power-up; nothing holding GPIO 0 or 2 in the wrong state.
- [ ] Analog with Wi-Fi on? Use GPIO 32–39 only.
- [ ] Buttons on 34–39 need an external pull-up (10 kΩ to 3V3).
- [ ] No 5 V signals into any pin; level-shift them.
- [ ] The ADC is non-linear near 0 V and above ~3.1 V; use `analogReadMilliVolts()` for calibrated readings.
- [ ] Random resets when Wi-Fi starts = power brownout; add a big capacitor and a better cable.
- [ ] "Failed to connect" when uploading = hold BOOT, tap EN.
- [ ] Avoid relays on 1, 3, 5, 14, 15: they twitch at boot.
- [ ] Heavy work in a tight loop? Add `delay(1)` or `yield()` so the watchdog doesn't reset you.

Sources

- ESP32 Series Datasheet, Espressif
- ESP32-WROOM-32E / 32UE Module Datasheet, Espressif

Pin functions, power numbers and peripheral counts follow Espressif's datasheets; board-level details (LED pin, regulator, typical free heap) are typical for DevKit V1 boards and can vary by maker.